
A1

Java Basics and Problem Solving

**Modern Code
MC (MTD.ba VZ)
SS 2025**

Submission via Moodle

24 Points

This assignment focuses on basic Java programming concepts, problem-solving with pseudocode, and understanding the development environment. You will create several Java programs that demonstrate fundamental programming skills.

Submission Requirements:

- Read the [Organizational Guidelines](#) on Moodle, and adhere to them!
 - Submit a PDF document containing your solution idea, code, and tests for each exercise
 - Include all Java source files (.java) in a ZIP archive
 - As tests include copies or screenshots of the console output
 - Use the naming convention: MC_A01_LASTNAME.pdf and MC_A01_LASTNAME.zip
-

Exercise 1 (4 points) Java - Personal Information Program

Create a Java program called `PersonalInfo.java` that displays your personal information. The program should:

- Declare variables for your name, age, and student ID
- Display this information using simple print statements
- Use appropriate data types for each variable
- Include comments explaining what each part of the code does

Expected Output:

```
1  My name is: [Your Name]
2  My age is: [Your Age]
3  My student ID is: [Your Student ID]
```

Exercise 2 (4 points) Java - Simple Calculator

Create a Java program called `SimpleCalculator.java` that performs basic arithmetic operations. The program should:

- Declare two integer variables with values of your choice
- Calculate and display the sum, difference, product, and quotient
- Handle division carefully (e.g. $3 / 5 = 0.6$)
- Display results in a clear, organized format

Exercise 3 (6 points) Pseudocode: Turtle Maze Navigation

Problem: A turtle needs to navigate through a maze to reach the exit. The turtle can only move forward, turn left, or turn right. The maze is represented as a grid where:

- # = Wall (turtle cannot move here)
- . = Empty space (turtle can move here)
- S = Start position, turtle always starts facing north (up direction)
- E = Exit position

Maze Layout:

```

    0 1 2 3 4 5 6
0 # E # # # # #
1 # . . . # . #
2 # . # . # . #
3 # . # . # . #
4 # # # . # . #
5 # S . . . . #
6 # # # # # # #
```

Your Task:

1. Write pseudocode for manually navigating the turtle through the maze. (4 points)
2. Desk test your pseudocode with the given maze layout for each step of the turtle's movement give the position and direction of the turtle after each step. Example (for a different maze): (2 points)

Line	Statement	row	col	direction
1	START	7	7	north
2	TURN LEFT	7	7	west
3	FORWARD	6	7	west
4	END	6	7	west

(You do not need to write a solution idea or test cases for this exercise, just the trace table is enough, as it is equal to the pseudocode.)

PSEUDOCODE RESTRICTIONS:

- START - begins the algorithm
- FORWARD - moves the turtle forward one step
- TURN LEFT - turns the turtle left by 90 degrees
- TURN RIGHT - turns the turtle right by 90 degrees
- END - ends the algorithm

Exercise 4 (10 points) Pseudocode: Number Guessing Game Logic

Problem: Design the logic for a number guessing game where:

- The computer thinks of a random number between 1 and 100
- The player tries to guess the number
- After each guess, the computer tells the player if their guess is too high, too low, or correct
- The game tracks the number of attempts

Your Task:

1. Write pseudocode for the complete number guessing game logic (7 points)
2. Simulate your pseudocode with a sample run (3 points)

PSEUDOCODE RESTRICTIONS:

- **START** - begins the algorithm
- **GENERATE RANDOMNUMBER(min, max)** - generates a random number between min and max
- **READ** - reads a number from the user
- **PRINT message** - displays a message to the user
- **IF condition THEN action** - conditional statement
- **ELSE IF condition THEN action** - alternative conditional statement
- **ELSE action** - default action when no condition is true
- **WHILE condition DO action** - loop while condition is true
- **REPEAT action UNTIL condition** - loop until condition is true
- **INCREMENT variable** - adds 1 to a variable
- **SET variable = value** - assigns a value to a variable
- **END** - ends the algorithm (or the statement with **END WHILE**, **END IF**)
- **Conditions:** You can use =, >, <, to compare variables

Example Interaction:

```
1      I'm thinking of a number between 1 and 100.
2      Enter your guess: 50
3      Too high! Try again.
4      Enter your guess: 25
5      Too low! Try again.
6      Enter your guess: 37
7      Correct! You guessed it in 3 attempts.
```

Simulation Example: Create a trace table showing line-by-line execution of your pseudocode. Choose your own secret number and trace through a few sample guesses (at least 3 guesses) For each line of pseudocode executed, show the current variable values.

Example Trace Table Format (simple counter example):

Line	Statement	count	sum	Condition
1	START	-	-	-
2	SET count = 1	1	-	-
3	SET sum = 0	1	0	-
4	WHILE count $\leq$ 3	1	0	$1 \leq 3 = \text{true}$
5	ADD count TO sum	1	1	-
6	INCREMENT count	2	1	-
7	WHILE count $\leq$ 3	2	1	$2 \leq 3 = \text{true}$
8	ADD count TO sum	2	3	-
9	INCREMENT count	3	3	-
10	WHILE count $\leq$ 3	3	3	$3 \leq 3 = \text{true}$
11	WHILE count $\leq$ 3	3	3	$4 \leq 3 = \text{false}$
